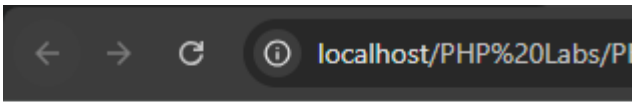
Lab-13

Q1: Implement a Vehicle Rental System
Objective: Apply abstraction and polymorphism.
Instructions:
● Create an abstract class Vehicle with a method getDetails().
● Create two child classes Car and Bike that implement getDetails().
● Create an object of both classes and display their details.

Code:

```
<?php
abstract class Vehicle {
    abstract public function getDetails();
}

class Car extends Vehicle {
    public function getDetails() {
        return "Car: 4 wheels, AC included";
    }
}

class Bike extends Vehicle {
    public function getDetails() {
        return "Bike: 2 wheels, Helmet included";
    }
}

$car = new Car();
$bike = new Bike();

echo $car->getDetails() . "<br>";
echo $bike->getDetails();
?>
```

Output:



Car: 4 wheels, AC included
Bike: 2 wheels, Helmet included

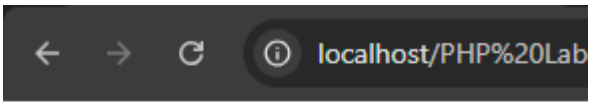
Q2:Create an abstract class BankAccount with:
○ An abstract method getBalance()
○ A concrete method deposit($amount)
● Create two child classes:
○ SavingsAccount

○ **CheckingAccount**
● **Implement the getBalance() method in each subclass.**
🔹 **Expected Output:**
Deposited: 1000
Balance: 1000

Code:

```
<?php
abstract class BankAccount {
    protected $balance = 0;

    abstract public function getBalance();

    public function deposit($amount) {
        $this->balance += $amount;
        echo "Deposited: $amount <br>";
    }
}

class SavingsAccount extends BankAccount {
    public function getBalance() {
        echo "Balance: " . $this->balance;
    }
}

class CheckingAccount extends BankAccount {
    public function getBalance() {
        echo "Balance: " . $this->balance;
    }
}

$account = new SavingsAccount();
$account->deposit(1000);
$account->getBalance();
?>
```

Output:



Deposited: 1000
Balance: 1000

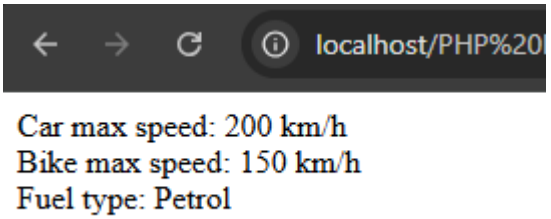
Q3:Create an abstract class Vehicle with:
○ **An abstract method maxSpeed()**
○ **A non-abstract method fuelType() returning "Petrol"**
● **Create two child classes:**

○ **Car**
○ **Bike**
● **Implement maxSpeed() in each subclass with different values.**
🔹 **Expected Output:**
Car max speed: 200 km/h
Bike max speed: 150 km/h
Fuel type: Petrol

Code:

```
<?php
abstract class Vehicle {
    abstract public function maxSpeed();

    public function fuelType() {
        return "Petrol";
    }
}

class Car extends Vehicle {
    public function maxSpeed() {
        return "Car max speed: 200 km/h";
    }
}

class Bike extends Vehicle {
    public function maxSpeed() {
        return "Bike max speed: 150 km/h";
    }
}

$car = new Car();
$bike = new Bike();

echo $car->maxSpeed() . "<br>";
echo $bike->maxSpeed() . "<br>";
echo "Fuel type: " . $car->fuelType();
?>
```

Output:

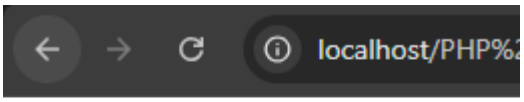


Q4:Create an abstract class Employee with:
○ Properties: name, salary
○ Abstract method calculateBonus()
○ Concrete method getDetails()
● Create subclasses:
○ Manager (bonus = 20% of salary)
○ Developer (bonus = 10% of salary)
● Implement calculateBonus() in each subclass.
🔹 Expected Output:
Manager Bonus: 2000
Developer Bonus: 1000

Code:

```
<?php
abstract class Employee {
    protected $name;
    protected $salary;

    public function __construct($name, $salary) {
        $this->name = $name;
        $this->salary = $salary;
    }

    abstract public function calculateBonus();

    public function getDetails() {
        return "Name: $this->name | Salary: $this->salary";
    }
}

class Manager extends Employee {
    public function calculateBonus() {
        return $this->salary * 0.20;
    }
}

class Developer extends Employee {
    public function calculateBonus() {
        return $this->salary * 0.10;
    }
}

$manager = new Manager("Rahul", 10000);
$developer = new Developer("Amit", 10000);

echo "Manager Bonus: " . $manager->calculateBonus() . "<br>";
echo "Developer Bonus: " . $developer->calculateBonus();
?>
```

Output:



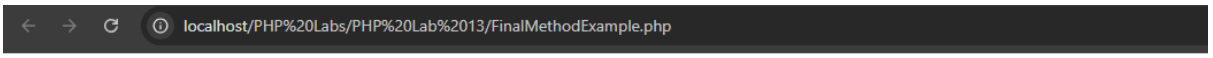
Manager Bonus: 2000
Developer Bonus: 1000

Q5:Create a class BaseClass with a final method showMessage()
● Create a subclass ChildClass and try to override showMessage()
● Run the code and observe the error.
🔹 Expected Error:Fatal error: Cannot override final method BaseClass::showMessage()

Code:

```
<?php
class BaseClass {
    final public function showMessage() {
        echo "This is a final method.";
    }
}

class ChildClass extends BaseClass {
    // This will cause error
    public function showMessage() {
        echo "Trying to override";
    }
}
?>
```

Output:



Fatal error: Cannot override final method BaseClass::showMessage() in **C:\xampp\htdocs\PHP Labs\PHP Lab 13\FinalMethodExample.php** on line **10**

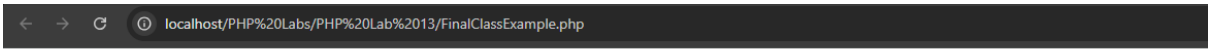
Q6:Understand how final prevents class inheritance.
● Create a final class SecuritySystem
● Try to create a subclass AdvancedSecuritySystem that extends SecuritySystem
● Run the code and observe the error.
🔹 Expected Error: Fatal error: Class AdvancedSecuritySystem cannot extend final class SecuritySystem

Code:

```
<?php
final class SecuritySystem {
    public function protect() {
        echo "Security Active";
```

```
    }
}

class AdvancedSecuritySystem extends SecuritySystem {
}
?>
```

Output:



Fatal error: Class AdvancedSecuritySystem may not inherit from final class (SecuritySystem) in **C:\xampp\htdocs\PHP Labs\PHP Lab 13\FinalClassExample.php** on line **9**

Q7:Create an abstract class Device with:

○ **A final method powerOn()**

○ **An abstract method specifications()**

● **Create a subclass Smartphone and implement specifications()**

● **Try to override powerOn() in Smartphone and observe the error.**

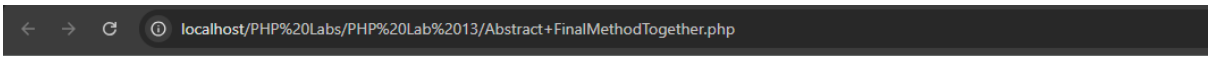
🔹 **Expected Output: Powering on the device... Smartphone specifications: 6GB RAM, 128GB Storage**

Code:

```
<?php
abstract class Device {

    final public function powerOn() {
        echo "Powering on the device... <br>";
    }

    abstract public function specifications();
}

class Smartphone extends Device {

    public function specifications() {
        echo "Smartphone specifications: 6GB RAM, 128GB Storage";
    }

    // Uncommenting below will cause error
    /*
    public function powerOn() {
        echo "Overriding powerOn";
    }
    */
}

$phone = new Smartphone();
$phone->powerOn();
$phone->specifications();
```

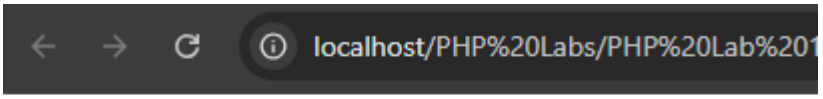
?>

Error:



Fatal error: Cannot override final method Device::powerOn() in **C:\xampp\htdocs\PHP Labs\PHP Lab 13\Abstract+FinalMethodTogether.php** on line **19**

Output:



Powering on the device...
Smartphone specifications: 6GB RAM, 128GB Storage